COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/4/2026	**DATE SUBMITTED:**	9/8/2026
NAME:	Cedric S. Fernandez	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 7.1Algorithms Pt1 **Sorting Algorithms Pt1**

Table 7-1. Array of Values for Sort Algorithm Testing

CODE

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;
void displayArr(int arr[]);

int main() {

    //create qan array with unsorted list wit h100 maximum size
    int unsortedArr[maxSize];
    // use time  (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for (int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }
    //display the randomly generated array
    displayArr(unsortedArr);
}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout<< '\n';
}
```

OUTPUT

```
61 22 4 60 87 71 84 23 75 80 44 88
 3 67 12 43 95 18 39 87 11 15 33 6
9 25 75 41 1 15 39 57 52 20 83 89
18 32 86 16 63 76 70 85 24 45 8 70
 3 23 11 66 31 75 94 56 48 68 86 7
4 71 28 36 91 48 86 87 48 41 39 66
 81 62 56 73 87 22 22 4 49 89 75 9
5 40 82 79 24 67 83 85 86 47 26 53
 28 38 19 78 70 34 87
```

OBSERVATION

Best Case - **O(N) For the best case the code would need to run through every item in the arrays and manually print everyone.**

Worst Case - **O(N) The program will have to look into every slot of the array to print it making it take longer if the array size is at a high value or N meaning it will have to manually do everything by step.**

Table 7-2. Bubble Sort Technique

CODE

main.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;
void displayArr(int arr[]);

int main() {

    //create qan array with unsorted list wit h100 maximum size
    int unsortedArr[maxSize];
    // use time  (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for (int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }
    //display the randomly generated array
    displayArr(unsortedArr);

//============= BUBBLE SORT CALL FUNCTION ================
    std::cout<<"\n bubble sort: \n";
    bubbleSort(unsortedArr, maxSize);
    displayArr(unsortedArr);
}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout<< '\n';
}
```

sortingAlgo.h

```cpp
#include <cstddef>
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>
//================BUBLE SORT============================
template<typename T>
void bubbleSort(T arr[], size_t arrSize){
    //step1: for i =0 to N-1 repeat inner loop
    for(int i =0; i < arrSize; i++){
        //step 2 inner loop that for K = i+1 to N-I repaeat
        for(int j = i+1; j < arrSize; j++){
            //step 3: swap if the arr[j] > arr[i]
            if(arr[i] > arr[j]){
                std::swap(arr[i],arr[j]);
            }
        }
    }
}
#endif
```

OUTPUT

Unsorted

```
24 0 85 94 2 67 25 20 28 35 64 81 4
6 46 24 98 77 9 75 71 96 95 82 87 5
4 95 80 92 45 83 33 24 85 44 99 29
33 8 28 75 83 64 12 27 66 82 56 69
29 24 71 39 10 30 80 65 48 32 88 53
 21 31 45 84 74 25 0 80 4 75 7 63 5
1 39 89 16 41 3 66 49 25 28 42 29 1
 6 53 10 77 87 85 37 56 2 91 52 27
66 38 89
```

Sorted

```
 bubble sort:
0 0 1 2 2 3 4 6 7 8 9 10 10 12 16 2
0 21 24 24 24 24 25 25 25 27 27 28
28 28 29 29 29 30 31 32 33 33 35 37
 38 39 39 41 42 44 45 45 46 46 48 4
9 51 52 53 53 54 56 56 63 64 64 65
66 66 66 67 69 71 71 74 75 75 75 77
 77 80 80 80 81 82 82 83 83 84 85 8
5 85 87 87 88 89 89 91 92 94 95 95
96 98 99
```

OBSERVATION

Best Case - O(N^2) for the best case if the array given is already sorted it would still need to run the for loop and the inner for loop because nothing checks if the swaps have already been done so it would still need to go through every line

Worst Case - O(N^2) for the worst case this would be if the array is all not in order and once more it would have to run the for loop and inner for loop over and over until every item is sorted

Table 7-3. Selection Sort Algorithm

CODE

main.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;
void displayArr(int arr[]);

int main() {

    //create qan array with unsorted list wit h100 maximum size
    int unsortedArr[maxSize];
    // use time  (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for (int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }
    //display the randomly generated array
    displayArr(unsortedArr);

//============= SELECTION SORT CALL FUNCTION ================
    std::cout<<"\n selection sort: \n";
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout<< '\n';
}
```

OUTPUT

Unsorted

```
42 93 24 49 58 79 91 18 25 26 47 46
 71 2 7 92 35 44 61 50 35 45 65 69
55 57 84 89 2 68 18 49 98 81 66 71
27 76 56 17 78 53 29 30 94 97 18 74
 91 96 95 91 17 12 62 69 78 89 82 5
3 95 58 26 62 64 63 60 27 44 79 64
0 48 84 69 47 30 93 29 14 29 51 28
82 5 99 77 57 57 70 58 21 29 66 61
48 48 37 7 11
```

Sorted

```
 selection sort:
0 2 2 5 7 7 11 12 14 17 17 18 18 1
8 21 24 25 26 26 27 27 28 29 29 29
 29 30 30 35 35 37 42 44 44 45 46
47 47 48 48 48 49 49 50 51 53 53 5
5 56 57 57 57 58 58 58 60 61 61 62
 62 63 64 64 65 66 66 68 69 69 69
70 71 71 74 76 77 78 78 79 79 81 8
2 82 84 84 89 89 91 91 91 92 93 93
 94 95 95 96 97 98 99
```

sortingAlgo.h

```
#include <cstddef>
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>
```

```
//============= SELECTION SORT CALL FUNCTION =================
template<typename T>
//array, current position, size of array
int routineSmallest(T arr[], int K, const int arrSize){
    int position = K;
    T smallestElem = arr[K];
        for(int j = K+1; j<arrSize; j++){
            if(arr[j] < smallestElem){
                smallestElem = arr[j];
                position = j;
            }
        }
        return position;
    }

template<typename T>
void selectionSort(T arr[], size_t arrSize){
    int POS, temp, pass;
        for(int i=0; i < arrSize; i++){
            POS = routineSmallest(arr, i,arrSize);
            //swapping:
            temp = arr[i];
            arr[i] = arr[POS];
            arr[POS] = temp;
            //for verification and counting of number of passes that was done
            pass++;
        }
    }
```

OBSERVATION

Best Case - O(N^2) for the best case regardless if it's already sorted the selectionSort function will still make the routineSmallest function to look through every array item just to check if the current item is really the smallest.

Worst Case - O(N^2) for the worst case even if unsorted the same will still happen and it will look through every item in the array taking the smallest and swapping it to the front making the best and worst case the same regardless of the situation.

Table 7-4. Insertion Sort Algorithm

CODE

main.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;
void displayArr(int arr[]);

int main() {

    //create qan array with unsorted list wit h100 maximum size
    int unsortedArr[maxSize];
    // use time  (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for (int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }
    //display the randomly generated array
    displayArr(unsortedArr);

//============= INSERTION SORT CALL FUNCTION ================
    std::cout<<"\n insertion sort: \n";
    insertionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);
}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout<< '\n';
}
```

sortingAlgo.h

```cpp
#include <cstddef>
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>
```

OUTPUT

Unsorted

```
35 85 15 79 40 96 2 35 60 26 2 44 5
8 56 17 27 80 59 25 20 55 88 88 37
80 62 66 76 41 89 39 62 35 14 17 8
74 74 24 6 73 88 68 17 72 43 47 9 1
5 59 20 36 81 42 23 34 89 18 25 61
40 98 10 66 67 14 34 17 46 55 13 66
 36 50 79 9 81 48 63 30 94 90 89 40
 24 57 73 35 80 44 74 96 81 41 64 1
8 91 66 14 14
```

Sorted

```
 insertion sort:
0 0 0 0 1 1 2 4 6 6 6 6 7 7 8 9 12
12 15 16 18 21 21 22 25 26 26 26 26
 27 28 28 29 33 34 34 34 35 35 35 3
6 36 37 38 39 40 41 42 42 43 44 44
45 49 50 50 54 55 57 58 60 61 62 65
 66 68 68 69 70 71 71 72 72 73 75 7
6 78 78 79 79 80 80 83 86 86 88 90
90 91 91 92 93 94 95 96 97 97 98 99
 99
```

```
47  //============= INSERTION SORT CALL FUNCTION ================
48  template<typename T>
49  void insertionSort( T arr[], size_t arrSize){
50      int K = 0, J, temp;
51
52      while(K < arrSize){
53          temp = arr[K];
54          J=K-1;
55
56          while(J >= 0 && temp <= arr[J]){
57              arr[J+1]= arr[J];
58              J--;
59          }
60
61          arr[J+1] = temp;
62          K++;
63      }
64
65  }
66  #endif
67
```

OBSERVATION

Best Case - O(N) for the best case if all the items in the array are already sorted then the program will stop immediately thanks to the condition of temp <= arr[j] making it so that the while loop also doesn't run so in the end it will only look through the array once.

Worst Case - O(N^2) The worst case would be when all high numbers are on the far right side of the array instead of the left side making it so that every element needs to be compared and shifted back making it so that for every N times the loops runs the same will also happened for the inner loop

SUPPLEMENTARY

Pseudocode of Algorithm

START

Create an array A with 100 elements
Create an array called votes
Create an array called candidateName
Store the five candidate names

Generate 100 random values from 1 to 5
Store the values in array A

Display the unsorted array

Sort array A using insertion sort

Display the sorted array

FOR each candidate from 1 to 5
  Search the entire array for the candidate number
  Count how many times the candidate number appears
  Store the count in votes
END FOR

Display each candidate's name and vote count

Set Candidate 1 as the current winner

FOR each candidate from 2 to 5
  IF the candidate's vote count is greater than
    the current winner's vote count
     Set the candidate as the new winner
  END IF
END FOR

Display the winning candidate's name and vote count

END

Screenshot of Algorithm Code

Random vote generator

```
//Generates the random vote 1-5
for (int i = 0; i < maxSize; i++){
    unsortedArr[i] = rand()% 5 + 1;
}
```

Insertion sort

```
std::cout<<"\n insertion sort: \n";
insertionSort(unsortedArr, maxSize);
displayArr(unsortedArr);
```

```
#include <cstddef>
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>
```

```
template<typename T>
void insertionSort( T arr[], size_t arrSize){
    int K = 0, J, temp;

    while(K < arrSize){
        temp = arr[K];
        J=K-1;

        while(J >= 0 && temp <= arr[J]){
            arr[J+1]= arr[J];
            J--;
        }

        arr[J+1] = temp;
        K++;
    }

}
#endif
```

Vote counter

```
for (int candidate = 1; candidate <= 5; candidate++) {
    votes[candidate] = linearSearchAR(maxSize, candidate, unsortedArr);
}
// Display the number of votes for each candidate
```

Winner identifier

```
for (int candidate = 2; candidate <= 5; candidate++) {
    if (votes[candidate] > votes[winner]) {
        winner = candidate;
    }
}
```

OUTPUT TESTING

Output Console Showing Sorted Array

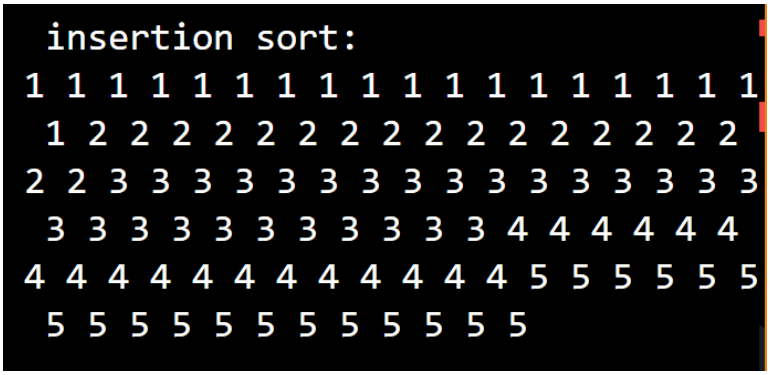
```
unsorted array
4 4 1 1 3 3 4 3 1 3 5 2 3 1 1 3 2 3 5 3 2 2 4 3 5 4 4 2 2 2 3 3 1 4 5 1 4 5 5 4 5 5 2 5 4 5 1 1 3 2 1 5 3 1 2 3 5 3 3 3 1 5 1 3 5 4 4 2 2 3 3 5 3
1 2 3 4 1 2 2 4 4 4 1 3 5 2 3 2 1 1 4 5 3 4 3 2 5 3 1

 insertion sort:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

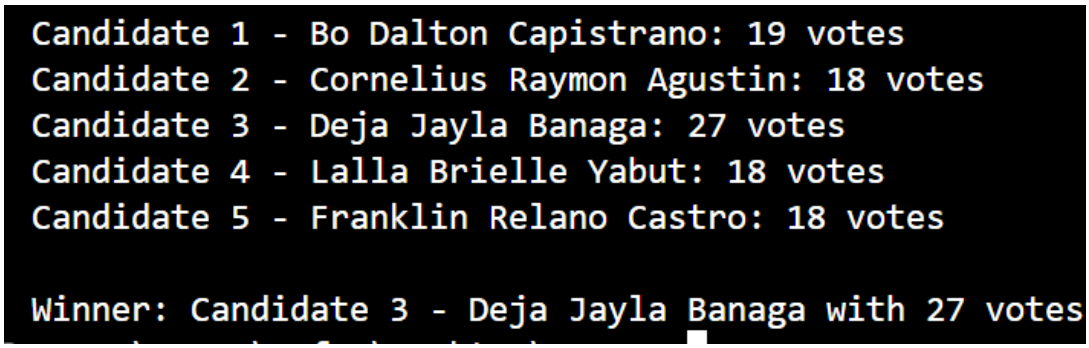
COMPUTER ENGINEERING
QUEZON CITY

Manual Count

- 1 = 19
- 2 = 18
- 3 = 27
- 4 = 18
- 5 = 18



Count Result of Algorithm



QUESTION:

Was your developed vote counting algorithm effective? Why or why not?

- The developed vote counting algorithm I made is effective because it successfully stored and counted the results and votes for each candidate without tinkering with the original value and by the end also successfully displayed the total count for each candidate and their total vote and also declared who the winner is.

CONCLUSION

For this activity I learned how sorting algorithms can help efficiently organize unsorted data and also learn the different kinds of sorting algorithms and how searching algorithms that were done in our last activity can be modified so that they not only find the value but count the instances a designated item is found it was also through this activity that I manage to reuse some of the code that was given to us into order to make an array that can store 100 int and making that the values it initializes is only 1-5 through the modulo operator while also making use of sorting algorithms to arrange the array so it can be

counted easily and also searching algorithms modifying it in a way so that it will count the instances that specific item appeared so that in the end it can show who has the most votes received and the amount each candidates got as shown on our supplementary activity.

For how I did during this activity I would like to think I did well in combining the sorting and searching algorithms and also modifying the searching algo although for my areas of improvement I kind of wished I knew more on how efficient each sorting and searching algorithms are based on the situation of the program and also the fact that I need to improve my ability to design and how to make a algorithm/pseudo code before trying to make the program.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of content, Grammar correction, Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to explain function concepts and how to properly use them, along with correcting my grammar for proper explanation and how to do circular Linked list.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.